

Unifying Simulation and Visualization: An Architectural Roadmap for Integrating World Data with Procedural City Generation

Executive Summary

Modern procedural content generation (PCG) systems are capable of producing outputs of immense scale and complexity, from intricate visual environments to deep, data-rich world simulations.¹ A significant challenge, however, arises when these systems operate in isolation. This report addresses a critical disconnect observed in many development pipelines: the

"Aesthetic-Data Divide," where a visually sophisticated procedural city generation system (CityGen) operates independently of a world data simulation system (WorldSim), resulting in game worlds that are visually impressive but lack a coherent, underlying identity. A city described by the simulation as an impoverished industrial hub may appear visually as a gleaming, modern metropolis, creating a fundamental break in plausibility and undermining the potential for emergent narrative.

The proposed solution is a fundamental architectural shift away from disconnected, single-purpose generators toward a unified, dynamic ecosystem. This report outlines a comprehensive, code-centric roadmap for the full integration of WorldSim and CityGen based on a **decoupled, event-driven architecture (EDA)**. This model establishes a central **Message Bus** as the communication backbone, allowing the two systems to interact asynchronously as peers. WorldSim publishes events representing significant changes in its state (e.g., economic shifts, population growth), and CityGen subscribes to these events, translating abstract data into tangible visual characteristics.

Key recommendations detailed within this report include:

1. **Adoption of an Event-Driven Architecture:** Utilizing a Message Bus to ensure systems are loosely coupled, independently scalable, and capable of robust,

asynchronous communication.³ This pattern is demonstrably superior to alternatives like point-to-point connections or shared databases for creating flexible and extensible world systems.

2. **Establishment of a Formal Data Contract:** Defining strict, versioned data schemas for all inter-system communication using a high-performance binary serialization format such as Google's Protocol Buffers or FlatBuffers. This ensures reliable data exchange and optimizes real-time performance.⁵
3. **Creation of an "Aesthetic Mapping Layer":** Implementing a dedicated, rules-based module within CityGen responsible for interpreting simulation data and translating it into concrete generation parameters. This layer is the creative heart of the system, where design intent and artistic direction are encoded into logic, transforming statistical data into urban morphology, architectural style, and environmental storytelling.⁷
4. **Implementation of a Bidirectional Feedback Loop:** Architecting a communication channel from CityGen back to WorldSim, allowing visual-layer events (whether player-driven or procedurally generated) to have persistent, systemic consequences within the simulation. This closes the loop, enabling true emergence and creating a world that reacts to and evolves with the player's presence.⁹

The successful integration of these systems represents a transformative opportunity. It enables a shift from the generation of static, random content to the simulation of a living, breathing world where the visual environment is a direct and dynamic reflection of its underlying social, economic, and historical truths. This approach moves beyond simple replayability, laying the foundation for deeply emergent gameplay and procedural narratives where the world itself becomes the central character. This report provides the complete architectural blueprint and a prioritized, phased implementation strategy to achieve this goal.

Section 1: Analysis of the Current Decoupled Architecture

Before designing a unified framework, it is essential to thoroughly analyze the existing state of the systems and the inherent limitations of their decoupled nature. This analysis defines the problem space, highlights the missed opportunities for creating a more dynamic and believable world, and establishes the foundational requirements

that the new architecture must satisfy.

1.1. The "Aesthetic-Data Divide": Defining the Problem Space

The current development environment consists of two powerful but isolated systems, each excelling in its respective domain but failing to form a cohesive whole. This separation creates a fundamental conflict between the world's simulated identity and its visual representation, a phenomenon termed the "Aesthetic-Data Divide."

System Characterization:

- **WorldSim (The Source of Truth):** This is a "headless" simulation engine, meaning it operates without a direct visual output. Its primary function is to model and evolve abstract, high-level data representing the state of the world. This includes complex models for economics (GDP, primary industries, wealth distribution), demographics (population size, density, cultural values), and history (major events, conflicts, periods of prosperity).¹¹ The output of WorldSim is a rich, quantitative dataset that defines the "truth" of a city or region at any given moment. It is, in essence, a system for simulating the socioeconomic and historical forces that shape civilizations.¹⁴
- **CityGen (The Visual Representation):** This is a procedural content generation engine focused exclusively on the creation of 3D urban environments. It leverages a suite of algorithms—such as L-systems for road networks, shape grammars for buildings, and noise functions for terrain and asset placement—to produce visually complex and varied cityscapes.¹⁵ Its output is the tangible, explorable geometry of the game world.

The Core Problem: The disconnect arises because the parameters driving CityGen are not derived from the rich data produced by WorldSim. Instead, CityGen likely relies on a combination of designer-tuned presets and pseudo-random number generation to determine a city's layout, architectural style, and density.¹⁷ This results in a "plausibility gap": a city that

WorldSim defines as a technologically advanced, high-GDP metropolis might be visually rendered by CityGen with quaint, low-rise brick buildings, while a simulated agrarian town might inexplicably feature a dense grid of skyscrapers. This inconsistency fundamentally undermines the player's suspension of disbelief and negates the value of having a deep, underlying world simulation. The visual

environment fails to tell the story that the simulation has written.

1.2. Limitations of a Non-Integrated Approach: Lost Emergence and Narrative Potential

The failure to bridge the Aesthetic-Data Divide extends beyond mere visual inconsistency. It imposes critical limitations on the potential for creating a truly dynamic, evolving, and narratively rich game world.

- **Static and Unreactive Worlds:** A key benefit of simulation-based PCG is the ability to create worlds that react to player actions and evolve over time.¹⁹ In a decoupled architecture, the world remains fundamentally static from a visual perspective. A simulated economic collapse, a technological breakthrough, or a major plague within WorldSim will have no discernible impact on the city's appearance. The city the player explores at the end of the game is visually indistinguishable from the one they started in, regardless of the dramatic events that have unfolded in the underlying simulation.
- **Lack of Emergent Behavior:** Emergence in complex systems arises from the feedback loops between constituent parts. A non-integrated approach severs these loops. For example, a truly emergent system might see a resource boom in WorldSim lead to rapid, chaotic urban sprawl in CityGen. This new sprawl could, in turn, create traffic and pollution problems that are fed back into WorldSim, triggering a new set of social and economic challenges.²⁰ Without integration, this entire class of complex, unpredictable, and deeply engaging behavior is impossible. The systems can produce content, but they cannot create a self-regulating, evolving ecosystem.
- **Shallow Environmental Storytelling:** The most compelling game worlds are imbued with history and lore that players can discover through exploration.² A simulation's history provides a powerful source for this environmental storytelling.²² A city that endured a great fire 100 years ago should have a district of visibly newer architecture. A civilization that won a major war might have grand monuments, while one that suffered a crushing defeat might have ruins and cemeteries. When WorldSim and CityGen are disconnected, this entire historical context is lost. The environment becomes a generic backdrop rather than a tangible record of the world's past, robbing it of depth and meaning.²³

1.3. Establishing Core Requirements for a Unified System

To overcome these limitations and create a truly unified world system, the integration architecture must meet a set of stringent technical and design requirements. These requirements will serve as the guiding principles for the architectural framework proposed in the subsequent sections.

- **Data-Driven Control:** The primary and most fundamental requirement is that the statistical and historical data generated by WorldSim must become the principal input that controls and constrains the output of CityGen. Generation must be a function of simulation state.
- **Decoupling and Modularity:** The integration solution must not create a monolithic, tightly-coupled system. WorldSim and CityGen should remain as separate, independently testable, and independently developable modules.²⁴ This is crucial for long-term maintainability and allows specialized teams to work on each system without causing cascading failures.
- **Scalability:** The architecture must be designed to handle significant scale. This includes the ability to manage hundreds or thousands of unique cities, each with its own complex simulation state, and to accommodate the future addition of new systems (e.g., an AI Director, a weather system) without requiring a complete architectural redesign.
- **Bidirectional Feedback:** The architecture must not be a one-way street. It must support a communication channel allowing events and state changes within the visual CityGen layer (e.g., player actions, procedurally triggered events) to be fed back into WorldSim, influencing its future state. This is the key to enabling a truly interactive and emergent world.
- **Real-Time (or Near-Real-Time) Updates:** For the game world to feel alive and responsive, changes in the simulation must be propagated to the visual representation with a latency that is acceptable for the gameplay experience. While not every simulation tick needs to trigger a full city regeneration, significant events should be reflected visually in a timely manner.

Section 2: A Unified Architectural Framework for Data-Driven Generation

The selection of an appropriate integration architecture is the most critical decision in unifying the WorldSim and CityGen systems. The chosen pattern must not only facilitate data transfer but also embody the principles of decoupling, scalability, and bidirectionality established in the core requirements. A thorough evaluation of standard integration patterns reveals that an Event-Driven Architecture (EDA) built around a central Message Bus is the optimal solution for this complex, real-time environment.

2.1. Evaluating Integration Patterns

A systematic analysis of common software integration patterns highlights their respective strengths and weaknesses in the context of a dynamic game world simulation.

- **Point-to-Point Connection:** This pattern involves creating a direct, custom link between WorldSim and CityGen.³ While simple for two systems, it is fundamentally unscalable. Each system must have explicit knowledge of the other's API and location. Adding a third system, such as a Faction AI, would require new, bespoke connections to both existing systems, leading to a brittle "spaghetti architecture" that is difficult to maintain and extend. This tight coupling violates the core requirement of modularity.
- **Shared Database:** In this model, both systems would read from and write to a common, central database.³ This approach is fraught with peril for a real-time application. The database becomes a single point of failure and a significant performance bottleneck. Furthermore, it forces a rigid, unified data schema onto two systems with vastly different needs; WorldSim requires a schema optimized for transactional simulation updates, while CityGen needs data structured for efficient geometric generation. This pattern creates extreme coupling at the data layer and is ill-suited for high-performance, independent systems.
- **Service-Oriented Architecture (SOA) / Remote Procedure Call (RPC):** This pattern exposes the functionality of each system as a set of callable services.³ For instance, CityGen could make a synchronous call like `WorldSim.getEconomyData("alpha-01")`. While this is a viable pattern and useful

for specific, on-demand queries, relying on it as the primary integration method can lead to complex and fragile chains of blocking calls. If WorldSim needs to notify CityGen of an update, it must actively call a CityGen service. This requires WorldSim to know about all potential listeners, which again introduces a form of coupling and makes it difficult to broadcast state changes to multiple, unknown systems.

- **Event-Driven Architecture (EDA) / Message Bus:** This pattern, identified as the optimal solution, decouples systems entirely.³ Systems communicate indirectly by publishing "events" (messages describing a state change) to a central Message Bus. Other systems can "subscribe" to the types of events they are interested in without the publisher needing any knowledge of who or how many subscribers exist. This aligns perfectly with the Observer design pattern, a cornerstone of robust and scalable game development.²⁷ It directly supports the requirements for decoupling, scalability, and asynchronous communication, making it the ideal foundation for a dynamic world system.

2.2. Proposed Architecture: An Event-Driven Model with a Central Message Bus

The proposed architecture organizes the systems as independent producers and consumers of events, orchestrated by a high-performance, in-memory Message Bus. This central bus acts as the nervous system for the entire world simulation, ensuring that information flows efficiently and reliably without creating direct dependencies between components.

Architectural Components:

- **The Message Bus:** This is the central communication backbone. It is a logical component responsible for receiving messages from producers and routing them to the appropriate subscribers. For a high-performance game engine, this would be implemented as an in-memory message queue system, potentially leveraging multi-threading to handle high throughput. Technologies like ZeroMQ or a custom C++ implementation using thread-safe queues and condition variables are suitable candidates.²⁸
- **Event Producers:** Any system that generates state changes and publishes them as events. WorldSim is the primary producer, emitting events like CityEconomyUpdated, PopulationDemographicsChanged, or HistoricalEventOccurred. As the system evolves, CityGen and other game systems

will also become producers.

- **Event Consumers:** Any system that subscribes to specific event types to react to state changes. CityGen is the primary consumer, listening for events from WorldSim and using the data they contain to trigger or modify its generation processes.
- **Asynchronous Communication:** The communication is inherently asynchronous. When WorldSim publishes an EconomyUpdated event, its work is done. It does not block or wait for a response from CityGen.²⁸ It immediately continues its simulation loop. This is critical for performance, as visual generation and asset streaming in CityGen can be computationally expensive and time-consuming operations that should not stall the core world simulation.

2.3. System Responsibilities: Defining the Roles

This architecture enforces a clear separation of concerns, defining distinct and unambiguous roles for each major component. This clarity is essential for parallel development and long-term system health.

- **WorldSim - The Source of Truth:**
 - **Responsibility:** To model and evolve all non-visual, abstract world data.
 - **Interface:** Publishes events to the Message Bus whenever a significant, persistent change in its state occurs.
 - **Boundary:** It has zero knowledge of rendering, 3D geometry, meshes, or textures. It communicates exclusively in terms of its abstract data model (e.g., GDP, population count).
- **CityGen - The Visual Representation Engine:**
 - **Responsibility:** To translate abstract data into a tangible, 3D visual representation of the world.
 - **Interface:** Subscribes to relevant events from the Message Bus and uses their data payloads to drive its internal generation algorithms.
 - **Boundary:** It has zero knowledge of the internal logic of the economic, demographic, or historical models within WorldSim. It only understands the data contract of the events it consumes.
- **The Integration Layer - The Connective Tissue:**
 - **Components:** The Message Bus implementation and the formally defined Data Contracts (message schemas).

- **Responsibility:** To guarantee the reliable delivery of messages and to enforce the structure and semantics of the data being exchanged. It is the shared language that allows the decoupled systems to communicate effectively.

The following table provides a clear justification for the selection of the Event-Driven Architecture over the alternatives, evaluated against the project's core requirements.

Table 2.1: Comparison of Integration Architectures

Integration Pattern	Coupling	Scalability	Real-Time Capability	Fault Tolerance	Suitability for this Project
Point-to-Point	Very High	Poor	Poor (Blocking Calls)	Low (Single link failure)	Unsuitable
Shared Database	High (Data Layer)	Poor (DB Bottleneck)	Moderate	Low (Single point of failure)	Unsuitable
Service-Oriented (RPC)	Moderate	Moderate	Good (for queries)	Moderate	Viable, but suboptimal for broadcasts
Message Bus (EDA)	Very Low	Excellent	Excellent (Asynchronous)	High (Decoupled systems)	Optimal

Section 3: The Data Bridge: Defining the WorldSim to CityGen API and Data Contracts

With the event-driven architecture established, the next critical step is to define the precise "language" that the systems will use to communicate. This is accomplished by creating a formal **Data Contract**: a set of versioned, strictly-defined data structures for the messages passed over the bus. A well-designed data contract is the bedrock of a stable and maintainable distributed system, preventing ambiguity and ensuring that producers and consumers have a shared understanding of the information being

exchanged.

3.1. The "World State" Data Model: Structuring the Messages

The payload of each event must be a well-defined data structure that encapsulates a specific state change. These structures should be as lean as possible, containing only the data relevant to the event, to minimize serialization overhead and network traffic. All messages should contain metadata such as a unique city/region identifier and a timestamp to ensure proper context.

Example Event Payload Structures (Conceptual JSON for readability):

An event signaling a significant update to a city's economy would be structured as follows. This event is published periodically or when major economic thresholds are crossed.

JSON

```
{
  "eventId": "evt_a4b2c1d0",
  "eventType": "EconomyUpdatedEvent",
  "cityId": "alpha-01",
  "timestamp": 1672531200,
  "payload": {
    "gdp": 15000000000,
    "gdpPerCapita": 12000,
    "primaryIndustry": "Technology",
    "secondaryIndustry": "Finance",
    "unemploymentRate": 0.04,
    "wealthDistribution": {
      "giniCoefficient": 0.45,
      "incomeQuintiles": [0.05, 0.10, 0.15, 0.25, 0.45]
    }
  }
}
```

```
}
```

Similarly, an event related to demographic shifts would contain population-specific data.

JSON

```
{
  "eventId": "evt_e5f6g7h8",
  "eventType": "PopulationUpdatedEvent",
  "cityId": "alpha-01",
  "timestamp": 1672531200,
  "payload": {
    "totalPopulation": 125000,
    "populationDensity": 3500,
    "growthRate": 0.015,
    "demographics": {
      "ageBrackets": { "0-18": 0.25, "19-64": 0.60, "65+": 0.15 },
      "culturalValues": { "collectivism": 0.7, "riskAversion": 0.3, "tradition": 0.8 }
    }
  }
}
```

Crucially, these schemas must be **versioned**. For example, a future version of the `EconomyUpdatedEvent` might add a `techLevel` field. By versioning the schema (e.g., `EconomyUpdatedEvent_v2`), older consumers that don't understand the new field can safely ignore it, while newer consumers can take advantage of it. This prevents system-wide updates every time a data model is expanded.

3.2. API Design: A Hybrid Approach

While the primary mode of communication is event-driven broadcasting, a purely event-based system can be inefficient for certain use cases. Therefore, a hybrid approach combining event broadcasts with a request-reply pattern for specific

queries is recommended.

- **Event-Based for State Changes (Publish/Subscribe):** This is the default and primary communication pattern. WorldSim broadcasts significant state changes as they occur. This is a "push" model, ideal for keeping CityGen and other systems passively updated with the latest world state. It is highly scalable, as one event can be consumed by any number of subscribers.²⁸
- **Service-Based for On-Demand Queries (Request/Reply):** Occasionally, CityGen might need specific, historical, or calculated data that is not part of a standard event broadcast. For example, upon generating a new "historic district," it might need to query WorldSim for the city's economic state 100 years ago to influence building decay. Instead of a direct, blocking RPC call, this can be implemented over the message bus.
 1. CityGen publishes a QueryHistoricalDataRequest event containing the cityId and the desired time period.
 2. WorldSim subscribes to these request events. Upon receiving one, it retrieves the data.
 3. WorldSim then publishes a HistoricalDataResponse event, containing the requested data and a correlation ID to link it back to the original request.
 4. CityGen listens for the response with the matching correlation ID and uses the data.This pattern preserves the asynchronous, non-blocking nature of the architecture while allowing for specific, on-demand data retrieval.

3.3. Data Serialization and Performance Considerations

The format used to serialize these data structures into a byte stream for transmission over the Message Bus has a profound impact on performance, especially in a real-time game environment. While human-readable formats like JSON are excellent for development and debugging, they are too verbose and slow to parse for production use.³⁰ A high-performance, binary serialization format is required.

Recommended Formats:

The ideal choice is a schema-based binary format, which enforces the data contract at the serialization layer and offers superior performance.

- **Google Protocol Buffers (Protobuf):** An industry-standard format known for generating very compact binary payloads and providing fast serialization and

deserialization across multiple languages.⁵ It requires a parsing step to convert the binary data back into a usable object, but this process is highly optimized. Its strong typing and schema evolution features make it an excellent choice for maintaining the data contract.

- **FlatBuffers:** Another high-performance format from Google, FlatBuffers' key advantage is its "zero-copy" deserialization capability. Serialized data is structured in a way that it can be accessed directly from the memory buffer without any parsing or memory allocation.⁶ This makes it exceptionally fast for read-heavy workloads, which is precisely the primary use case for CityGen (consuming and reading data from WorldSim events). For this reason, **FlatBuffers is the primary recommendation for this architecture.**

The choice between them involves a trade-off: Protobuf generally produces slightly smaller payloads, while FlatBuffers offers significantly faster read access. Given that CityGen will be reading data far more often than WorldSim is writing it, the performance benefits of FlatBuffers' zero-copy approach are compelling.

Table 3.1: World State Data Dictionary (Excerpt)

This table serves as the definitive specification for the data contract. It is a critical piece of documentation that must be maintained and agreed upon by the teams developing both WorldSim and CityGen.

Parameter Path	Data Type	Description	Unit / Range	Update Frequency	Associated Event
Economy.gdp	float64	Gross Domestic Product of the city.	Currency units	On change > 5%	EconomyUpdatedEvent
Economy.primaryIndustry	string	The dominant economic sector (e.g., "Technology", "Heavy Industry",	Enum list	On change	EconomyUpdatedEvent

		"Tourism").			
Economy.we althDistributi on.giniCoeffi cient	float32	A measure of income inequality.	0 (perfect equality) to 1 (perfect inequality)	On change > 0.02	EconomyUp datedEvent
Population.to talPopulation	uint64	Total number of residents in the city.	Integer > 0	On change > 2%	PopulationU pdatedEvent
Population.p opulationDe nsity	float32	Population per square kilometer.	Float > 0	On change > 2%	PopulationU pdatedEvent
Population.d emographics .culturalValu es.tradition	float32	A normalized value representing the population's adherence to traditional values.	0 (progressive) to 1 (traditional)	On change > 0.05	PopulationU pdatedEvent
History.lastM ajorConflict	int32	Years since the last major war or civil unrest.	Integer >= 0	On event	HistoricalEve ntOccurred

Section 4: Translating Simulation to Visualization: A Practical Implementation Roadmap

With a robust architectural foundation and a well-defined data contract, the central challenge becomes the translation of abstract simulation data into compelling visual form. This is not a simple one-to-one mapping but a creative and technical process of interpretation. This section provides a practical roadmap for implementing this translation, centered around a conceptual component within CityGen called the "Aesthetic Mapping Layer."

4.1. The "Aesthetic Mapping" Layer: A Rules Engine for Interpretation

The Aesthetic Mapping Layer is the bridge between the analytical world of WorldSim and the visual world of CityGen. It is a dedicated software component within CityGen whose sole responsibility is to subscribe to events from the Message Bus and convert their data payloads into a set of concrete parameters that can be directly consumed by CityGen's procedural algorithms.

This layer is where the art direction and design intent of the project are encoded as executable logic. It functions as a rules engine, applying a series of transformations and heuristics to the incoming simulation data. For instance, it takes a raw GDP number and translates it into parameters like `averageBuildingHeight`, `skyscraperDensity`, and `materialQuality`. This approach draws heavily from techniques used in data-driven urban simulation and planning, where statistical and geospatial data are used to generate realistic 3D models of cities.⁷

In a modern game engine like Unreal Engine, this layer could be implemented as a high-level manager class or, more elegantly, as a top-level Procedural Content Generation (PCG) Graph. This graph would have custom nodes designed to ingest data from an external source (the Message Bus) and pipe the translated values into the rest of the generation graph that handles geometry and asset placement.³⁴

4.2. Economic Indicators → Urban Morphology

The economic state of a city is one of the most powerful drivers of its visual character. The Aesthetic Mapping Layer must contain rules to translate key economic indicators into the city's physical form.

- **GDP and Wealth:** A city's Gross Domestic Product and per-capita wealth are direct proxies for its level of development.
 - **High GDP/Wealth:** This should translate to a higher `averageBuildingHeight`, an increased probability of generating skyscrapers in the central business district, and a preference for high-end architectural styles using materials like glass, steel, and polished stone. Infrastructure would appear modern and well-maintained.

- **Low GDP/Wealth:** This results in smaller, lower-rise buildings, a prevalence of functional materials like brick, concrete, and corrugated metal, and the procedural application of visual decay effects like grime, rust, and cracked facades.
- **Primary Industry:** The dominant industry shapes the very function and layout of urban districts. This allows for the creation of visually distinct zones based on economic activity.³⁶
 - **'Technology' or 'Finance':** These industries would generate clean, modern office parks or dense clusters of towering skyscrapers, respectively. The road network would be a highly organized grid to facilitate efficient transport.³⁷
 - **'Heavy Industry':** This would trigger the generation of large factory complexes, smokestacks, warehouses, and dense districts of functional, low-cost worker housing nearby.
 - **'Tourism':** This would increase the probability of generating hotels, entertainment venues, parks, and well-preserved historic districts.

A conceptual C++ implementation within the Aesthetic Mapping Layer might look like this:

C++

```
// Forward declaration of CityGen's parameter struct
struct CityGenerationParameters;

// The AestheticMapper class consumes WorldSim events
class AestheticMapper {
public:
    // This method translates an economy event into generation parameters
    CityGenerationParameters mapEconomyToVisuals(const EconomyUpdatedEvent& event) {
        CityGenerationParameters params;

        // Map primary industry to architectural style set and road layout
        if (event.payload.primaryIndustry == "Technology") {
            params.buildingStyleSet = EBuildingStyle::ModernGlass;
            params.roadNetworkType = ERoadNetwork::Grid;
            params.parkGenerationChance = 0.4f;
        } else if (event.payload.primaryIndustry == "Heavy Industry") {
```

```

    params.buildingStyleSet = EBuildingStyle::IndustrialBrick;
    params.roadNetworkType = ERoadNetwork::Functional;
    params.proceduralEffects.smogDensity = 0.6f;
}

// Map GDP to building height and density
params.averageBuildingHeight = scaleValue(event.payload.gdp, MIN_GDP,
MAX_GDP, 15.0f, 250.0f);
params.skyscraperDensity = scaleValue(event.payload.gdpPerCapita,
MIN_GDP_PC, MAX_GDP_PC, 0.05f, 0.8f);

return params;
}

private:
// Helper function to scale a value from one range to another
float scaleValue(double value, double inMin, double inMax, float outMin, float outMax) {
    // Clamped linear interpolation
    double clampedValue = std::max(inMin, std::min(value, inMax));
    return outMin + (float)((clampedValue - inMin) / (inMax - inMin)) * (outMax -
outMin);
}
};

```

4.3. Population and Demographics → City Layout

The people who inhabit a city are the primary force shaping its layout and social fabric. Demographic data from WorldSim can be used to generate cities that feel socially and culturally authentic.

- **Population Density:** This is a key input for road network generation algorithms.
 - **High Density:** Favors the generation of tight, efficient grid systems, similar to those found in major metropolitan centers. This can be achieved by constraining L-system growth rules or by rewarding agent-based road builders for straight, interconnected paths.¹⁶
 - **Low Density:** Leads to more organic, sprawling layouts with winding roads and cul-de-sacs, characteristic of suburban or rural areas.

- **Wealth Disparity (Gini Coefficient):** This metric is a powerful tool for creating visually and socially distinct neighborhoods.
 - **High Gini Coefficient:** Triggers the generation of sharply contrasting districts. The rules engine would create zones for opulent, gated communities with large properties and manicured parks, often immediately adjacent to zones of dense, low-quality tenement housing with narrow alleys and minimal green space.
 - **Low Gini Coefficient:** Results in more uniform, homogenous neighborhoods, such as sprawling middle-class suburbs or consistent urban apartment blocks.
- **Cultural Values:** Abstract cultural metrics from the simulation can influence architectural details and public spaces.
 - **High 'Collectivism':** Increases the generation probability of large public squares, communal housing projects, public transportation hubs, and grand monuments to the state.
 - **High 'Individualism':** Favors the generation of single-family homes with private yards, sprawling road networks designed for personal vehicles, and commercial districts dominated by individual retail stores rather than large markets.
 - **High 'Tradition':** Could cause the generator to favor older, more ornate architectural styles and preserve historic-looking districts, even in a high-GDP city.

4.4. History and Events → Environmental Storytelling

A city is a living record of its past. By consuming historical events from WorldSim, CityGen can embed narrative and history directly into the environment, creating a world ripe for discovery.¹⁹

- **Past Conflicts:** An HistoricalEventOccurred message with a 'War' type from 50 years ago could trigger a "scarring" pass in CityGen. This pass would procedurally generate ruined buildings, overgrown fortifications, or memorial parks on the sites of simulated major battles.
- **Economic Cycles:** The history of economic booms and busts can create architectural layers. A boom 100 years ago might result in a district of ornate, Victorian-style buildings. A subsequent bust could mean that this "old town" is now surrounded by simpler, more functional structures, creating a visible timeline

in the city's architecture.

- **Natural Disasters:** A simulated earthquake in the distant past could lead to a specific city region having building codes that favor newer, seismically-resistant architectural styles, making it look visibly different from the rest of the city that was unaffected.

Table 4.1: Data-to-Aesthetic Mapping Rules (Excerpt)

This table provides a concrete specification for the Aesthetic Mapping Layer, serving as a vital communication tool between designers, artists, and engineers.

Simulation Data Input	Condition	CityGen Parameter Affected	Resulting Visual Effect
Economy.gdpPerCapita	>50000	materialSet	Use high-end materials (glass, steel, marble).
Economy.gdpPerCapita	<10000	proceduralDecay.intensity	Increase grime, rust, and graffiti effects.
Economy.primaryIndustry	== "Heavy Industry"	zoneType	Generate industrial districts with factories, smokestacks.
Population.populationDensity	>8000/km2	roadNetwork.gridBias	Generate a tight, grid-like road network.
Population.populationDensity	<1000/km2	roadNetwork.organicBias	Generate a sprawling, organic road network with cul-de-sacs.
Economy.wealthDistribution.giniCoefficient	>0.6	district.variance	Create extreme contrast between wealthy and poor districts.

Population.demographics.culturalValues.tradition	>0.8	buildingStyleSet	Prioritize older, more ornate architectural styles.
History.lastMajorConflict	< 20 years	proceduralScatter.ruins	Add ruined buildings and rubble piles throughout the city.

Section 5: Achieving Dynamic Realism: Bidirectional Communication and System Evolution

A truly dynamic and emergent world requires more than a one-way flow of data from simulation to visualization. It demands a feedback loop where actions and events within the visual layer have meaningful, persistent consequences in the underlying simulation. The proposed event-driven architecture is uniquely suited to facilitate this bidirectional communication, transforming CityGen from a passive renderer into an active participant in the world's evolution.

5.1. Closing the Loop: City Events Influencing the World Simulation

In this enhanced model, CityGen is not merely a consumer of events; it also becomes a producer. It is responsible for observing significant occurrences within its domain—whether initiated by the player, by AI agents, or by its own procedural sub-systems—and publishing them as events onto the Message Bus. WorldSim then subscribes to these events and updates its internal state accordingly.

This creates a complete cycle: WorldSim state changes affect the visual city, and events in the visual city affect the WorldSim state.

Example Events Published by CityGen:

- **DistrictDestroyedEvent:** Published when a player (or a procedural disaster like a fire) destroys a significant portion of a city district. The payload would include the cityId, the type of district (e.g., 'industrial', 'commercial'), and the percentage of destruction.

- **MajorInfrastructureBuiltEvent:** Published when the player completes the construction of a major, game-changing piece of infrastructure, like an airport or a power plant. The payload would detail the infrastructure type and its operational capacity.
- **ResourceNodePlacedEvent:** If CityGen's terrain generation algorithms procedurally place a new resource deposit (e.g., a mine), it publishes an event to inform WorldSim of this new economic opportunity.
- **PopulationDisplacedEvent:** Triggered by in-game events like monster attacks or natural disasters that cause citizens to flee a specific area.

5.2. Patterns for Bidirectional Synchronization

Achieving this feedback loop requires a robust synchronization strategy. The goal is to maintain consistency between the two systems without re-introducing the tight coupling that the architecture was designed to avoid.⁹

- **State Synchronization (Rejected):** This pattern involves one system periodically sending its entire state to the other for reconciliation. This is highly inefficient, does not scale, and is unsuitable for a real-time, decoupled model.³⁸ It would require CityGen to send a complete representation of its geometry to WorldSim, which is unnecessary and computationally prohibitive.
- **Event Sourcing / Message Passing (Recommended):** This is the chosen approach and a natural extension of the EDA. Systems do not exchange their full state; they only communicate *deltas*—events that describe what has changed.⁹ CityGen publishes a `FactoryDestroyed` event. WorldSim, upon receiving this message, trusts that the event is true and updates its own state by reducing the city's industrial output and increasing unemployment. It does not need to query CityGen for the factory's status. This method is highly efficient, scalable, and maintains the strict decoupling of the systems.³⁹ Real-time data exchange is managed through the continuous, low-latency flow of these discrete event messages.⁴⁰

5.3. Use Case: How a Newly Generated Port in CityGen Updates Trade Routes in WorldSim

To illustrate the power of this bidirectional feedback loop, consider the full lifecycle of an event that originates in the simulation, is visualized, and then feeds back into the simulation to cause further change.

1. Simulation Trigger (WorldSim → CityGen):

- WorldSim's simulation runs, and due to favorable conditions, the economy of a coastal city, "Portia," surpasses a key GDP threshold.
- WorldSim publishes an EconomyUpdatedEvent to the Message Bus with Portia's new, higher GDP and its coastal location attribute.

2. Visual Generation (CityGen Reaction):

- CityGen's Aesthetic Mapping Layer, a subscriber to EconomyUpdatedEvent, receives the message for Portia.
- It processes the data through its rules engine. A rule states: IF gdp > X AND location == 'Coastal' AND hasPort == false THEN generatePort = true.
- This triggers CityGen's port generation sub-routine. It carves out a harbor, places dock assets, warehouses, cranes, and connects them to the existing road network. The visual landscape of Portia is now permanently altered.

3. Feedback Event (CityGen → WorldSim):

- Upon successful completion of the port geometry, CityGen acts as a producer. It constructs and publishes a MajorInfrastructureBuiltEvent to the Message Bus.
- The event payload contains: { "cityId": "Portia", "infrastructureType": "Seaport", "cargoCapacity": 50000 }.

4. Simulation Update (WorldSim Reaction):

- WorldSim contains a "Trade Simulation" module that subscribes to MajorInfrastructureBuiltEvent.
- It receives the event for Portia. The module updates its internal representation of the world's logistics network, adding a new seaport node at Portia with the specified capacity.
- The trade simulation re-calculates optimal trade routes. New, profitable sea routes are now established between Portia and other cities, which were previously only accessible via more expensive land routes.⁴²

5. Emergent Consequences (WorldSim → CityGen):

- The new trade routes significantly boost Portia's economy and the economies of its new trading partners. WorldSim calculates a new, even higher GDP for Portia and updates the economic states of the other cities.
- WorldSim publishes a new batch of EconomyUpdatedEvent messages for all affected cities.

- CityGen receives these new events. For Portia, the higher GDP might trigger the generation of a wealthier commercial district near the new port. For its trading partners, the economic boost might cause their own cities to expand and develop.

This complete cycle demonstrates a true emergent feedback loop. A change initiated by the abstract simulation led to a visual change, which in turn created a new opportunity within the simulation, leading to further abstract changes and subsequent visual evolution. The player is no longer just observing a procedurally generated world; they are witnessing a simulated world that visibly grows and adapts based on a coherent set of internal logics. If the player were to destroy the port, the reverse would happen: CityGen would publish a `DistrictDestroyedEvent`, `WorldSim` would sever the trade routes, the economy would decline, and the city's visual appearance would degrade over time, reflecting the new economic reality. This makes player actions deeply meaningful and their consequences systemic and persistent.

Section 6: Advanced Topics and Future-Proofing

The proposed event-driven architecture provides a robust and scalable foundation. However, to achieve state-of-the-art results and ensure the system remains extensible for years to come, it is prudent to consider advanced techniques and future-proofing strategies. These enhancements can further improve the plausibility of the generated content, manage the performance demands of a massive world, and provide developers with the tools needed to effectively build and debug such a complex system.

6.1. Enhancing Plausibility with Machine Learning and Agent-Based Models

While a rules-based Aesthetic Mapping Layer is effective, its output is ultimately constrained by the creativity and foresight of the rules designed by humans. Advanced AI and simulation techniques can be integrated to produce more organic, surprising, and realistic results.

- **Inverse Procedural Modeling with Machine Learning:** Instead of relying solely

on hand-crafted rules, machine learning models can be trained on real-world data (e.g., satellite imagery, GIS data, urban planning datasets) to learn the complex relationships between demographic/economic data and urban layouts.⁴³ A Generative Adversarial Network (GAN), for example, could be trained to generate plausible city road networks or building footprints. In this scenario, the data from a

WorldSim event would not be fed into a simple rules engine, but would instead be used as a conditioning input for the trained ML model, guiding it to generate a visually distinct style that is statistically consistent with the input data.

- **Agent-Based Simulation within CityGen:** The current model assumes a top-down generation approach. A more sophisticated, bottom-up method would be to use agent-based simulation within CityGen itself.²⁰ Instead of a single generator placing buildings, thousands of individual "developer agents" could operate within the city. Each agent would be given goals and would make decisions based on local conditions and global data from WorldSim events. For example, a "commercial developer agent" would seek out land with high traffic (from WorldSim's transit data) and high wealth concentration, bidding against "residential agents" for desirable plots. This can lead to far more organic and emergent city growth patterns than a purely top-down approach.⁴⁵
- **Large Language Models (LLMs) for Narrative Flavor:** While WorldSim provides the quantitative data, LLMs can be used to generate qualitative, narrative flavor text that reflects the world state.¹ For instance, upon receiving an `EconomyUpdatedEvent` indicating a recession, a dedicated "Narrative Service" could use an LLM to generate in-game newspaper headlines ("Factories Close as Recession Bites"), NPC dialogue snippets ("Times are tough since the mines shut down"), or quest descriptions that are contextually aware of the current state of the world.⁴⁶

6.2. Scalability and Performance: Managing Large Worlds

A world with hundreds of simulated cities and millions of entities presents significant performance challenges. The architecture must be designed with scalability in mind from the outset.

- **Asynchronous and Parallel Generation:** The processing of events by CityGen must be non-blocking. When an event arrives, it should be placed into a local

work queue. The actual regeneration of city geometry should be handled by a separate background thread or a job system. This ensures that the main game loop and UI remain responsive, even during a large-scale city update.

- **Simulation Level of Detail (LOD):** It is computationally wasteful for WorldSim to simulate every city in the world at the highest level of fidelity at all times. A simulation LOD system should be implemented. Cities that are far from the player or are not central to the current narrative can be run on a simplified, low-fidelity simulation model that requires fewer resources. When the player approaches or interacts with a city, WorldSim can seamlessly switch it to the full, high-fidelity simulation, publishing more detailed and frequent events.
- **Distributed Simulation and the Message Bus:** The choice of a Message Bus architecture makes the system naturally suited for distribution across multiple processes or physical servers.⁴⁷ For a truly massive world, the simulation load can be partitioned. For example, one WorldSim server instance could be responsible for simulating "Continent A," while another handles "Continent B." Because they both publish to and subscribe from the same logical Message Bus, they can still interact (e.g., simulating trade between continents) while operating as independent services. This allows for horizontal scaling to meet nearly any performance requirement.

6.3. Tooling and Debugging for a Data-Driven PCG Pipeline

A complex, asynchronous, data-driven system can be notoriously difficult to debug. Investing in specialized tooling is not a luxury but a necessity for developer productivity and system stability.

- **Event Loggers and Visualizers:** A critical tool is a "bus sniffer" that logs every single event published to the Message Bus to a searchable database. This log should be paired with a visualization tool that can replay the event stream. A developer trying to debug why a city looks wrong could use this tool to scrub back in time, inspect the exact data payload of the WorldSim event that triggered the generation, and see precisely what information CityGen was working with.
- **Simulation Time Scrubbing:** The development team needs tools to manipulate the simulation's state for testing purposes. This includes the ability to "time scrub"—rewinding WorldSim to a specific point in its history and re-broadcasting the state events for that moment. This allows for deterministic testing of how CityGen's Aesthetic Mapping Layer responds to specific historical data without

having to re-run the entire simulation.

- **Data Contract Validation and Schema Registry:** To prevent integration errors, an automated validation system should be implemented. Before a new version of WorldSim or CityGen is deployed, its event production/consumption logic should be tested against a central "Schema Registry." This registry contains the canonical definitions of all event data structures and their versions. These automated tests can catch bugs caused by mismatched data schemas (e.g., WorldSim starts sending GDP as a string instead of a float) before they ever reach production.
-

Section 7: Conclusion and Strategic Recommendations

The integration of a deep world simulation with a visual city generator represents a paradigm shift from creating static game levels to cultivating dynamic, living worlds. The "Aesthetic-Data Divide"—the gap between a world's simulated identity and its visual representation—is a significant barrier to creating believable and immersive experiences. The proposed event-driven architecture, centered on a decoupled Message Bus, provides a robust, scalable, and future-proof solution to this challenge.

By treating the WorldSim and CityGen systems as independent peers that communicate through a well-defined, asynchronous event stream, this framework achieves the core requirements of modularity and data-driven control. The introduction of the **Aesthetic Mapping Layer** provides a crucial bridge between abstract data and visual artistry, allowing designers to encode their intent into a rules engine that translates simulation outputs into tangible urban forms.

Furthermore, the architecture's support for **bidirectional communication** is its most transformative aspect. By enabling events in the visual layer to feed back into and persistently alter the world simulation, it creates the potential for true emergence. Player actions gain systemic weight, and the world evolves in a coherent, logical response. This elevates the player from a mere visitor to an active agent of change within a complex adaptive system.

To successfully implement this ambitious vision, a phased, iterative approach is strongly recommended. Attempting a "big bang" integration is fraught with risk; a structured roadmap that delivers value at each stage is essential for managing

complexity and building momentum.

Table 7.1: Prioritized Implementation Roadmap

Phase	Key Objectives	Core Tasks	Success Metrics
Phase 1: Architectural Foundation	Establish the core communication infrastructure and data contracts.	1. Select and implement a high-performance, in-memory Message Bus. 2. Define the initial Data Contract schemas (e.g., using FlatBuffers) for key events (EconomyUpdated, PopulationUpdated). 3. Create basic "hello world" producers and consumers in both WorldSim and CityGen to validate the communication pipeline. 4. Implement an initial version of the Event Logger for debugging.	- WorldSim can successfully publish an event to the bus. - CityGen can successfully subscribe to and receive that event. - Event payloads are correctly serialized and deserialized. - The entire process is logged for inspection.
Phase 2: Unidirectional Data Flow	Implement the WorldSim to CityGen data flow to achieve initial data-driven visuals.	1. Develop the first version of the Aesthetic Mapping Layer within CityGen. 2. Implement 2-3 simple data-to-visual mappings (e.g., GDP → average building height; Population Density → road network type). 3. Refactor WorldSim to publish	- A city with a high simulated GDP is visibly taller and denser than a city with a low GDP. - Changes made to the simulation state are reflected in newly generated cities. - The Aesthetic Mapping Layer correctly translates event data into

		EconomyUpdated and PopulationUpdated events when significant state changes occur. 4. Connect the systems and generate the first cities whose basic morphology is driven by simulation data.	CityGen parameters.
Phase 3: Bidirectional Feedback Loop	Close the loop by enabling CityGen to influence WorldSim, creating a dynamic, interactive world.	1. Define the Data Contracts for events originating from CityGen (e.g., DistrictDestroyedEvent). 2. Implement the logic in CityGen to detect and publish these events (e.g., triggered by player actions). 3. Implement subscriber modules within WorldSim to consume these events and update the simulation state accordingly (e.g., reduce industrial output after a DistrictDestroyedEvent). 4. Test a full feedback cycle: a WorldSim change causes a CityGen change, which in turn causes a new WorldSim change.	- A player destroying a factory in CityGen results in a logged drop in GDP in WorldSim. - The subsequent EconomyUpdatedEvent from WorldSim causes a visible degradation of the city's appearance over time. - The world feels reactive and persistent.
Phase 4: Advanced Systems & Optimization	Enhance plausibility, performance, and developer tooling for the mature system.	1. Research and prototype ML-based generators (Inverse Procedural Modeling) for key visual	- The system can handle a world with 100+ fully simulated and visually represented cities at

		elements like road networks. 2. Implement a Simulation LOD system in WorldSim to manage performance in a large world. 3. Develop advanced debugging tools, such as a visual event replayer and simulation time-scrubbing controls. 4. Profile and optimize the entire pipeline, focusing on serialization overhead and generation hotspots.	interactive frame rates. - New visual styles can be integrated by training ML models instead of only writing new rules. - Developers have a robust suite of tools to diagnose and resolve integration issues efficiently.
--	--	--	--

By following this strategic roadmap, the development team can systematically de-risk a complex architectural undertaking while progressively unlocking the immense potential of a truly unified and dynamic procedural world.

Works cited

1. Procedural generation - Wikipedia, accessed July 21, 2025, https://en.wikipedia.org/wiki/Procedural_generation
2. Diving Into Procedural Content Generation, With WorldEngine - Smashing Magazine, accessed July 21, 2025, <https://www.smashingmagazine.com/2016/03/procedural-content-generation-introduction/>
3. Integration Patterns - Download Center - Microsoft, accessed July 21, 2025, <https://download.microsoft.com/download/a/c/f/acf079ca-670e-4942-8a53-e587a0959d75/intpatt.pdf>
4. www.confluent.io, accessed July 21, 2025, [https://www.confluent.io/learn/event-driven-architecture/#:~:text=Driven%20Architecture%20Works-,Event%2Ddriven%20architecture%20\(EDA\)%20is%20a%20software%20design%20pattern,the%20system%20drive%20the%20flow.](https://www.confluent.io/learn/event-driven-architecture/#:~:text=Driven%20Architecture%20Works-,Event%2Ddriven%20architecture%20(EDA)%20is%20a%20software%20design%20pattern,the%20system%20drive%20the%20flow.)
5. fastest c++ serialization? - Stack Overflow, accessed July 21, 2025, <https://stackoverflow.com/questions/3637581/fastest-c-serialization>
6. Benchmarking Eight Serialization Formats in C and C++ (JSON, BSON, CBOR, flexbuffers, msgpack, TOML, XML, YAML) : r/cpp - Reddit, accessed July 21, 2025,

- https://www.reddit.com/r/cpp/comments/1drz3eg/benchmarking_eight_serialization_formats_in_c_and/
7. A Data-Driven Interactive System for Smart Urban Planning and Design - ResearchGate, accessed July 21, 2025, https://www.researchgate.net/publication/386064580_A_Data-Driven_Interactive_System_for_Smart_Urban_Planning_and_Design
 8. A Procedural Modeling Approach for Ecosystem Services and Geodesign Visualization in Old Town Pocatello, Idaho - MDPI, accessed July 21, 2025, <https://www.mdpi.com/2073-445X/11/8/1228>
 9. Bidirectional Sync: What Is It & How Does It Improve Collaboration, accessed July 21, 2025, <https://readme.com/resources/bidirectional-sync>
 10. What is Bidirectional Communication? - AT-Automation, accessed July 21, 2025, <https://www.at-automation.nl/en/knowledge-base/bidirectional/>
 11. Procedural City Generation Beyond Game Development - acm sigspatial, accessed July 21, 2025, https://www.sigspatial.org/wp-content/uploads/special-issues/10/2/08-SIGSPATIAL_Newsletter_July_2018_Kim.pdf
 12. Simulations and games in economics education - Wikipedia, accessed July 21, 2025, https://en.wikipedia.org/wiki/Simulations_and_games_in_economics_education
 13. Economics Simulation Game: The Impact of Economics Simulation Games on Business Decision Making - FasterCapital, accessed July 21, 2025, <https://fastercapital.com/content/Economics-Simulation-Game--The-Impact-of-Economics-Simulation-Games-on-Business-Decision-Making.html>
 14. Urban Sim Models: A Concise Essential Guide - Number Analytics, accessed July 21, 2025, <https://www.numberanalytics.com/blog/urban-sim-models-essential-guide>
 15. (PDF) Citygen: An Interactive System for Procedural City Generation - ResearchGate, accessed July 21, 2025, https://www.researchgate.net/publication/357658334_Citygen_An_Interactive_System_for_Procedural_City_Generation
 16. A Survey of Procedural Techniques for City Generation, accessed July 21, 2025, https://www.citygen.net/files/images/Procedural_City_Generation_Survey.pdf
 17. Procedural Content Generation for video games, a friendly approach, accessed July 21, 2025, <https://www.levelup-gamedevhub.com/en/news/procedural-content-generation-for-video-games-a-friendly-approach/>
 18. Procedural Content Generation via Generative Artificial Intelligence - arXiv, accessed July 21, 2025, <https://arxiv.org/html/2407.09013v1>
 19. Procedural Content Generation - Game AI Pro, accessed July 21, 2025, http://www.gameaipro.com/GameAIPro2/GameAIPro2_Chapter40_Procedural_Content_Generation_An_Overview.pdf
 20. Procedural City Modeling - The CCL, accessed July 21, 2025, <https://ccl.northwestern.edu/2004/ProceduralCityMod.pdf>
 21. Simulated Economy (1) | Jason Fantl, accessed July 21, 2025,

- [https://jasonfantl.com/posts/Simulated-Economy-\(1\)/](https://jasonfantl.com/posts/Simulated-Economy-(1)/)
22. Procedurally Generated History: building a game ecosystem through autoplay - FDG 2015, accessed July 21, 2025, http://www.fdg2015.org/papers/fdg2015_paper_13.pdf
 23. Generating Procedural Narrative - Nonadecimal Creative, accessed July 21, 2025, <https://nonadecimal.com/site/procedural-narrative/>
 24. Understanding Integration Architecture: A Comprehensive Guide ..., accessed July 21, 2025, <https://www.ardog.com/knowledge-hub/integration-architecture>
 25. Event-Driven Architecture in Game Development: Unity & GameMaker, accessed July 21, 2025, <https://medium.com/@ahmadrezakml/event-driven-architecture-in-game-development-unity-gamemaker-c76915361ff0>
 26. Integration Patterns in Software Development - Architecture ..., accessed July 21, 2025, <https://www.architectureandgovernance.com/applications-technology/integration-patterns-in-software-development/>
 27. Game Programming Patterns - Medium, accessed July 21, 2025, <https://medium.com/@apoorvmishra05/game-programming-patterns-9e4a877dc942>
 28. Message Queues - System Design - GeeksforGeeks, accessed July 21, 2025, <https://www.geeksforgeeks.org/system-design/message-queues-system-design/>
 29. event handling - EventBus in C++ - Code Review Stack Exchange, accessed July 21, 2025, <https://codereview.stackexchange.com/questions/283817/eventbus-in-c>
 30. JSON vs Binary Serialization - That One Game Dev : r/programming - Reddit, accessed July 21, 2025, https://www.reddit.com/r/programming/comments/1bidw2b/json_vs_binary_serialization_that_one_game_dev/
 31. Would you recommend JSON/CSV/Other for data storage in games? : r/gamedev - Reddit, accessed July 21, 2025, https://www.reddit.com/r/gamedev/comments/11yk8h7/would_you_recommend_json_csv_or_other_for_data_storage/
 32. C++ Serialization Performance - Stack Overflow, accessed July 21, 2025, <https://stackoverflow.com/questions/321619/c-serialization-performance>
 33. felixguendling/cpp-serialization-benchmark: Comparison of C++ Serialization Libraries for Graph Data - GitHub, accessed July 21, 2025, <https://github.com/felixguendling/cpp-serialization-benchmark>
 34. Procedural Content Generation Overview | Unreal Engine 5.6 Documentation, accessed July 21, 2025, <https://dev.epicgames.com/documentation/en-us/unreal-engine/procedural-content-generation-overview>
 35. Custom PCG Elements - Unreal Engine Public Roadmap, accessed July 21, 2025, <https://portal.productboard.com/epicgames/1-unreal-engine-public-roadmap/c/101-custom-pcg-elements>
 36. FlorianLance/City-procedural-modeling: A random ... - GitHub, accessed July 21, 2025, <https://github.com/FlorianLance/City-procedural-modeling>

37. Coding a procedural city generator - Godot Forums, accessed July 21, 2025, <https://godotforums.org/d/19821-coding-a-procedural-city-generator>
38. Game server synchronization of large amounts of data in a battle, accessed July 21, 2025, <https://engineering.monstar-lab.com/en/post/2021/02/09/Game-server-Synchronization/>
39. What Is Bidirectional Communication? Definition - CoreTigo, accessed July 21, 2025, <https://www.coretigo.com/glossary/bidirectional-communication/>
40. Real-Time Multiplayer Synchronization - Lark, accessed July 21, 2025, https://www.larksuite.com/en_us/topics/gaming-glossary/real-time-multiplayer-synchronization
41. 66. Real-Time Data Synchronization: Methods, Use Cases, and Tool Selection - Medium, accessed July 21, 2025, <https://medium.com/@ilakk2023/real-time-data-synchronization-methods-use-cases-and-tool-selection-b990ab7144d4>
42. Simulating Trade Between Cities : r/gamedev - Reddit, accessed July 21, 2025, https://www.reddit.com/r/gamedev/comments/jlvhqk/simulating_trade_between_cities/
43. Procedural City Layout Generation with GANs - Maxwell Forbes, accessed July 21, 2025, <https://maxwellforbes.com/posts/procedural-map-generation-with-gans/>
44. Data-driven Urban Building Energy Modeling with Machine Learning in Satom (CH) - Idiap Publications, accessed July 21, 2025, <https://publications.idiap.ch/publications/show/5242>
45. An agent-based approach to procedural city generation ..., accessed July 21, 2025, <https://arxiv.org/abs/2211.01959>
46. Procedural Content Generation in Games: A Survey with Insights on Emerging LLM Integration - arXiv, accessed July 21, 2025, <https://arxiv.org/html/2410.15644v1>
47. Simulation Architectures in Complex Systems - CloudAstra, accessed July 21, 2025, <https://cloudastra.co/blogs/simulation-architectures-in-complex-systems>